Министерство Образования и исследований

Государственный Университет Молдовы

Кафедра информатики

Отчет:

По дисциплине Java Script

Лабораторная работа No2

Tema: Реализация базовых методов работы с массивами

Выполнил:

Чубару Максим, gr. IA2503

Проверил:

N.Calin

Кишинев, 2026

Цель работы

Целью данной работы является изучение принципов реализации базовых методов работы с массивами с использованием колбэков: `forEach`, `map`, `filter`, `find`, `some`, `every`, `reduce`, а также закрепление навыков написания функций с корректным возвратом результатов и обработкой граничных случаев.

Теоретическая часть

Формулировка задачи

В рамках лабораторной работы требовалось реализовать набор функций для работы с массивами без использования встроенных методов JavaScript. Основная задача заключалась в создании универсальных и переиспользуемых функций, принимающих колбэк в качестве аргумента.

Также необходимо было:

- реализовать различные способы вывода массива;
- создать универсальный механизм перебора элементов;
- реализовать аналоги стандартных методов массивов.

Описание цели и основных этапов работы

В ходе выполнения лабораторной работы были выполнены следующие этапы:

1. Реализация функций вывода массива:
   - `printArray`
   - `printArray1`
2. Создание универсальной функции:
   - `forEach` — для обхода массива и вызова колбэка
3. Реализация функций обработки массива:
   - `map` — преобразование элементов
   - `filter` — фильтрация элементов
   - `find` — поиск первого подходящего элемента
   - `some` — проверка наличия хотя бы одного элемента
   - `every` — проверка всех элементов
4. Дополнительно:
   - `reduce` — свертка массива в одно значение

Практическая часть

Краткое описание особенностей реализации

1. **Функции вывода (`printArray`, `printArray1`)**
   Были реализованы с использованием цикла `for`, так как требовался доступ к индексу и значению элемента. Отличие между ними заключается только в формате вывода.

```
function printArray(array){
    for(let i = 0; i<array.length; i++){
        console.log(`Element ${i}: value ${array[i]}`);
    }
}

function printArray1(array){
    for(let i = 0; i<array.length; i++){
        console.log(`${i}: ${array[i]}`);
    }
}
```

2. **Функция `forEach`**
   - Использует цикл `for`
   - Вызывает колбэк для каждого элемента
   - Передает: `element`, `index`, `array`
   - Ничего не возвращает

```
function forEach(array, callback){
    for(let i = 0; i<array.length; i++){
        callback(array[i], i, array);
    }
    return;
}
```

3. **Функция `map`**
   - Создает новый массив
   - Результат каждого вызова колбэка добавляется в новый массив
   - Длина результата равна исходной

```
function map(array, callback){
    const newArr = [];
    for(let i = 0; i<array.length; i++){
        newArr.push(callback(array[i], i, array));
    }
    return newArr;
}
```

4. **Функция `filter`**
   - Проверяет условие через колбэк
   - Добавляет элементы в новый массив только если возвращается `true`

```
function filter(array, callback){
    const newArr1 = [];
    for(let i = 0; i<array.length; i++){
        if(callback(array[i], i, array)){
            newArr1.push(array[i]);
        }
    }
    return newArr1;
}
```

5. **Функция `find`**
   - Возвращает первый элемент, удовлетворяющий условию
   - Прерывает выполнение при нахождении

```
function find(array, callback){
    for(let i = 0; i<array.length; i++){
        if(callback(array[i], i, array)){
            return array[i];
        }
    }
}
```

6. **Функция `some`**
   - Возвращает `true`, если найден хотя бы один подходящий элемент
   - Завершается досрочно при первом совпадении

```
function some(array, callback){
    for(let i = 0; i<array.length; i++){
        if(callback(array[i], i, array)){
            return true;
        }
    }
    return false;
}
```

7. **Функция `every`**
   - Проверяет, что все элементы соответствуют условию
   - Завершается при первом `false`

```
function every(array, callback){
    for(let i = 0; i<array.length; i++){
        if(!callback(array[i], i, array)){
            return false;
        }
    }
    return true;
}
```

8. **Функция `reduce`**
   - Использует аккумулятор
   - Учитывает наличие или отсутствие `initialValue`
   - Обрабатывает граничные случаи (например, пустой массив)

```javascript
function reduce(array, callback, initialValue){
    let start;
    let accumulator;
    if(array.length === 0 && initialValue === undefined){
        return;
    }

    if(initialValue === undefined){
        start = 1;
        accumulator = array[0]
    }
    else{
        start = 0;
        accumulator = initialValue;
    }

    for(let i = start; i<array.length; i++){
        accumulator = callback(accumulator, array[i], i, array);
    }
    return accumulator;
}
```

Вывод

В ходе выполнения лабораторной работы были изучены принципы работы с массивами и колбэками в JavaScript. Реализация собственных версий методов позволила лучше понять их внутреннюю логику.

Были закреплены навыки:

- работы с функциями высшего порядка;
- передачи колбэков;
- обработки массивов без встроенных методов;
- работы с граничными случаями.

Ссылка на репозиторий Git:
https://github.com/maximcv246-a11y/JS-Lab2

Ответы на контрольные вопросы

1. В чем преимущества использования колбэков при работе с массивами?

- Повышают гибкость кода
- Позволяют переиспользовать функции
- Упрощают обработку данных
- Делают код более декларативным

---

2. Какие проблемы могут возникать при использовании колбэков и как их избежать?

Проблемы:

- Сложность чтения (callback hell)
- Потеря контекста
- Ошибки в логике при передаче неправильных типов данных

Решения:

- Использовать стрелочные функции
- Разбивать код на небольшие функции
- Чтобы их избежать, необходимо проверять входные параметры, использовать `typeof` и `Array.isArray`, а также обрабатывать граничные случаи и при необходимости приводить данные к нужному типу.

---

3. Как реализовать функции map, filter, find, some, every и reduce без встроенных методов массивов?

Все функции реализуются через цикл `for`:

- `map` → создаем новый массив и добавляем результат колбэка
- `filter` → добавляем элемент только если условие истинно
- `find` → возвращаем первый подходящий элемент
- `some` → возвращаем `true` при первом совпадении
- `every` → возвращаем `false` при первом несоответствии
- `reduce` → накапливаем результат в аккумуляторе

Список использованных источников

1. Официальная документация JavaScript — https://developer.mozilla.org
2. JavaScript.info — https://javascript.info
3. Учебные материалы по дисциплине «Веб-программирование» — https://github.com/MSU-Courses/javascript_typescript